

Gradient Boosting

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain how gradient boosting chains simple models together to correct each other's mistakes
- Compare gradient boosting's sequential approach with random forest's parallel approach
- Calculate residuals and combine model outputs using the learning rate hyperparameter
- Apply `GradientBoostingRegressor` in code and interpret its performance against a single decision tree

2. Detailed Explanation

What is gradient boosting?

Gradient boosting is an ensemble learning technique: a way of combining multiple machine learning models so their combined prediction beats any single model's prediction. It isn't a brand-new algorithm by itself. It's a systematic method for chaining together simpler models (most commonly decision trees) so each new model corrects the mistakes of the ones before it. Gradient boosting can be built on top of almost any base algorithm, and it's one of two "named" ensemble strategies in common use — the other being random forest.

The name "boosting" reflects the mechanism: every round boosts the performance of the system by attacking the error left behind by the previous round.

Random forest vs. gradient boosting

There are two broad ways to combine multiple models into an ensemble, and understanding the contrast makes gradient boosting easier to grasp.

Random forest (parallel ensembling) trains multiple decision trees independently and in parallel, each on a different portion of the data. Every tree makes its own prediction, and the final decision comes from majority voting – if four out of five trees vote positive, the ensemble predicts positive. Most individual trees will be correct, so a majority vote is more reliable than any single tree.

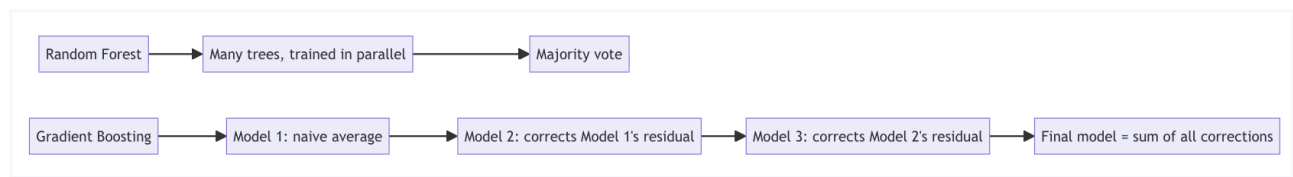
Gradient boosting (sequential ensembling) builds one model at a time, in sequence. A first, very simple model is trained. Its error is measured, and a second model is built specifically to reduce that error. A third model reduces the error left over after the second model, and so on. Each new model is bound to the objective of correcting the error of the model right before it.

The key differences:

Aspect	Random forest	Gradient boosting
Training order	Parallel, independent trees	Sequential – each model depends on the previous one
Final prediction	Majority vote across trees	The last model, which incorporates all prior corrections

Base algorithm	Always decision trees	Algorithm-agnostic (trees, SVMs, logistic/linear regression, or a mix)
Data per round	Each tree trains on a sample	Every model trains on the full training set, but on a transformed target (the residual)

Random forest and gradient boosting are the only two ensemble techniques with a fixed, well-known formulation. A developer is free to design custom ensembles beyond these two. For example, a developer could build a decision tree first, then improve its residual with a linear regression model instead of another decision tree, following the same sequential "boosting" pattern. Such combinations have no fixed name; they're experimental setups the developer designs.



The core mechanism: residuals, correction trees, and the learning rate

Gradient boosting follows a repeatable five-step loop:

1. Start with a simple base model. The first model, often called the weak model, makes a naive, non-trainable prediction. For a regression problem, this is typically just the average of the target values in the training data, assigned to every instance.
2. Compute the residual. The residual (the gap between what actually happened and what the model predicted) for each training instance is $\text{residual} = \text{actual} - \text{predicted}$.

3. Fit a new model to the residual, not the original target. A new model (commonly a decision tree) trains using the same input features as before, but with the residual from the previous round as the new target variable. It learns which feature values are associated with large positive or large negative residuals, so it can propose an adjustment.
4. Combine the new model's output with the previous prediction, scaled by a learning rate. The previous model is never discarded. Its prediction is kept, and the new model's output is added on top, scaled down by the learning rate (denoted η , or α):
$$\text{new_prediction} = \text{previous_prediction} + \eta \times \text{new_model_output}$$
5. Repeat. Steps 2–4 repeat for a fixed number of rounds. Each round produces a fresh, smaller residual and a fresh model trained on it. The number of rounds — equivalently, the number of models in the ensemble — is itself a hyperparameter, analogous to "number of trees" in a random forest.

After m rounds, the overall formula for the final model is:

$$\text{Final model} = M_1 + \eta \times DT_1 + \eta \times DT_2 + \dots + \eta \times DT_m$$

where M_1 is the initial naive average-based model, and DT_i is the decision tree trained on the input features and the residual left over from round $i - 1$.

Because the residual is literally the gap between actual and predicted values, gradient boosting works directly with that raw residual rather than a squared error. It does not square the residual before using it as the new target. Squaring errors is a separate practice used purely for reporting, to avoid negative values in a summary statistic.

Watch out for: boosting happens only on the training data. Residuals are computed on the training set at every round. The test data plays no role until the very end, when the final combined model (after all rounds are complete) is applied to the test set to measure real-world performance.

Worked example: predicting salary from experience

A tiny dataset of five employees ties the mechanism together:

Experience (years)	Salary
1	3
2	4
3	5
4	7
5	8

Round 0 (Model M1). The naive prediction is the average salary for every employee:

$\text{Average} = (3 + 4 + 5 + 7 + 8) / 5 = 5.4$. This isn't a trained model in any real sense — it's a starting point that shows how much gradient boosting improves on it later.

Residual of M1 (actual – predicted):

Experience	Actual	Predicted (M1)	Residual
1	3	5.4	-2.4
2	4	5.4	-1.4

3	5	5.4	-0.4
4	7	5.4	+1.6
5	8	5.4	+2.6

Building the first correction tree (DT1). A decision tree trains with experience as the input and the M1 residual as the new target – the actual salary column isn't touched here. This is a regression problem because the residual is continuous. The tree finds a split point:

- For `experience ≤ 3` (residuals -2.4, -1.4, -0.4): average = -1.4
- For `experience > 3` (residuals +1.6, +2.6): average = +2.1

DT1 isn't the final "model 2" yet – it only produces an adjustment value to combine with M1.

Combining with the learning rate to get M2. With $\eta = 0.5$, DT1 contributes only half of its adjustment:

$$M2 = M1 + \eta \times DT1$$

- For `experience ≤ 3`: $M2 = 5.4 + 0.5 \times (-1.4) = 4.7$
- For `experience > 3`: $M2 = 5.4 + 0.5 \times (2.1) = 6.45$

These predictions (4.7 for the first three employees, 6.45 for the last two) sit noticeably closer to the actual salaries than the flat 5.4 from M1. An actual salary of 3 is now predicted as 4.7 instead of 5.4, and an actual salary of 8 is now predicted as 6.45 instead of 5.4.

Continuing the sequence. A fresh residual is computed from M2's predictions, and a second correction tree, DT2, trains on experience and this new residual. Combining it the same way produces Model 3: $M3 = M1 + \text{eta} \times \text{DT1} + \text{eta} \times \text{DT2}$. This process — compute residual, fit a tree to the residual, add it in with the learning rate — repeats for as many rounds as the number-of-estimators hyperparameter specifies. Each round's residual shrinks relative to the previous round, so the ensemble's combined prediction keeps getting closer to the actual values.

At each step, the newly-added tree is trained specifically to explain the leftover residual using the input feature. Where the previous prediction was too high, the new tree learns to push it down; where it was too low, the tree learns to push it up. Because the model is never allowed to make the full jump in one step — the learning rate scales the contribution — the ensemble converges gradually instead of overshooting.

The learning rate (eta) hyperparameter

Eta controls how much of each new correction tree is allowed to influence the combined prediction at every round. It's common to essentially all machine learning algorithms that learn iteratively, including neural networks and deep neural networks — not something unique to gradient boosting.

- Too small (e.g., $\text{eta} = 0.01$): the model makes very slow, cautious progress. Even after many rounds, the combined prediction may still be far from optimal. The model eventually reaches a good solution, but it may take an impractically long time — an extreme illustrative comparison is training that should take 10 minutes instead taking 24 hours.
- Too large (e.g., $\text{eta} = 1$ or 2): the model tries to reach the optimal solution too quickly and can overshoot it, landing on a worse solution and never settling into the best possible model. There's no guarantee of good performance in this regime.

- A balanced, moderate value gives the best trade-off between training time and final performance. Training should start out cautious and continue only until error stops decreasing, at which point it can stop.

The typical usable range for learning rate is roughly 0.01 to about 1–2. The default value used by Python machine learning libraries such as scikit-learn, when no learning rate is specified, is 0.1 — which tends to sit in the balanced zone.

Hyperparameters in general

A hyperparameter is a variable that takes part in a model's overall formula or setup but isn't learned as part of the training process itself. Its value isn't known in advance; only a rough range is known, and the exact value is found experimentally rather than mathematically derived. Hyperparameters are part of the experimental setup, not part of the trained model's internal parameters.

Examples covered so far:

- Number of estimators / rounds — how many models (trees) are chained together in the sequence, analogous to "number of trees" in a random forest.
- Learning rate (eta / alpha) — how strongly each new model's correction is applied.
- Maximum tree depth — how deep each individual decision tree in the sequence is allowed to grow.

There's no fixed formula for choosing the ideal learning rate; it's tuned experimentally by trying a range of values. Libraries such as scikit-learn provide functions that automate the search for a good hyperparameter value.

Worked example: car resale price and mean squared error

A second toy dataset illustrates gradient boosting on car mileage versus resale price. The example is framed around markets like the United States, where reselling used cars — including higher-end cars such as a Tesla being resold at a reduced price — is very common. Well-maintained roads and greater consumer purchasing power make trading up common there, unlike markets where low-cost, first-hand cars dominate.

Mileage (thousand km)	Price (\$ thousand)
20	18
45	14.5
60	12
80	9.5
100	8
130	5.5

As mileage increases, resale price decreases.

Round 0. The naive starting prediction is the average price across all six cars: $\text{Average} = (18 + 14.5 + 12 + 9.5 + 8 + 5.5) / 6 = 11.25$.

Mean squared error (MSE) tracks performance across rounds:

$$\text{MSE} = (1 / n) \times \sum (\text{actual} - \text{predicted})^2$$

Differences are squared and summed across all data points, then divided by the number of data points ($n = 6$ here).

Progress across rounds. The same procedure as the salary example is followed: compute the residual, fit a decision tree between mileage and the residual, and combine the tree's output with the running prediction using the learning rate. Error drops sharply round over round:

- Round 0 MSE ≈ 17.23 (naive average-only prediction)
- Round 1 MSE ≈ 7.60
- Round 2 MSE ≈ 3.74

Plotting mileage against price shows the round 0 prediction as a flat horizontal line, since everyone gets the same average price. The round 1 prediction line starts tilting toward the true downward-sloping relationship between mileage and price. The round 2 prediction line sits noticeably closer to the true linear-looking drop in price as mileage increases, particularly at the extremes of the mileage range.

Code walkthrough: building gradient boosting by hand

Before reaching for a ready-made estimator, the same salary/mileage-style boosting procedure can be implemented directly in code, round by round. The essential library imports:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import tree
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, root_mean_squared_error,
r2_score
```

The single feature column (mileage) is reshaped so the algorithm sees it as

`number_of_instances × number_of_features`, the shape scikit-learn expects for input features:

```
X = mileage.reshape(-1, 1) # reshape a 1-D feature array into 2-D (rows, 1
                             feature)
y = price                    # actual price values, no reshaping needed
```

Round 0 assigns the mean price to every row, and a small history table (a pandas DataFrame) tracks the round number, current prediction, and MSE at each step:

```
eta = 0.5
pred = np.full_like(y, y.mean()) # round 0: predict the mean for every
row
history = pd.DataFrame({"round": [0], "prediction": [pred.copy()],
                        "mse": [mean_squared_error(y, pred)]})
```

Rounds 1 and 2 then run manually in a loop: compute the residual against the current prediction, fit a fresh decision tree — referred to here as a "stump", a shallow correction tree — on the feature and the residual, and update the running prediction by adding the learning-rate-scaled output of the new tree:

```
from sklearn.tree import DecisionTreeRegressor

for round_num in [1, 2]:
    residual = y - pred
    stump = DecisionTreeRegressor(max_depth=1) # a "stump" is a depth-1
decision tree
    stump.fit(X, residual) # tree trained on (features,
residual), not (features, y)
    pred = pred + eta * stump.predict(X) # boosted update: previous prediction
+ eta * correction
    mse = mean_squared_error(y, pred)
    # ...append round, prediction, mse to history
```

Running this manually confirms the same pattern seen in the worked-by-hand example: MSE falls from its round-0 value to a noticeably lower value after round 1, and lower

again after round 2. This manual, round-by-round construction is only for illustration — in practice, no one hand-codes 100 rounds of boosting; a dedicated estimator handles it directly.

GradientBoostingRegressor on a larger dataset

A larger, synthetic dataset of 150 car records demonstrates gradient boosting at a realistic scale, comparing different learning rates and a single decision tree.

Feature and target generation. Four columns were generated for the 150 synthetic cars: mileage (random values between 5 and 150), age (years), a condition score (higher values indicate better condition), and a noise column. The noise column is a random, uninformative feature not used to construct the target. Resale price was built as a linear combination of the meaningful features plus randomness:

```
price = 22 - 0.09 * mileage - age_weight * age + 0.9 * condition_score
price = price + np.random.uniform(0, 1.5, size=150) # add noise so price
isn't purely formulaic
price = np.clip(price, 2, None) # floor the price at 2,
no upper cap
```

Here 22 acts as a constant/bias term. Mileage carries a negative weight (−0.09) since higher mileage should reduce price; age is also negative since an older car should be worth less; condition score is positive (0.9) since a better-condition car should be worth more. A small amount of noise (0 to 1.5) is added so price isn't a perfectly deterministic function of the features, and price is floored at a minimum of 2 with no maximum limit.

Train/test split. The 150-row feature matrix and price target were split into training and test portions with `train_test_split` from `sklearn.model_selection`, so gradient boosting fits only on training data and its real performance is measured on held-out test data:

```
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

Comparing learning rates. `GradientBoostingRegressor` (from `sklearn.ensemble`) ran three times — a very small value (0.01), a moderate value (0.1), and a large value (1.0) — while keeping the number of rounds and tree depth fixed:

```
results = []
for eta in [0.01, 0.1, 1.0]:
    gb = GradientBoostingRegressor(
        n_estimators=100, learning_rate=eta, max_depth=2, random_state=42
    )
    gb.fit(X_train, y_train)
    preds = gb.predict(X_test)
    results.append((eta, r2_score(y_test, preds)))
```

R^2 (the coefficient of determination) measures how much of the variance in actual price is captured by predicted price — the maximum possible value is 1:

Learning rate	R^2
0.01	0.418
0.10	0.802
1.00	0.565

The moderate learning rate (0.1) gives by far the best result. The very small learning rate hasn't converged within 100 rounds, and the very large learning rate has overshoot the optimal model — both extremes confirm the learning-rate trade-off explained earlier.

Gradient boosting vs. a single decision tree. With learning rate fixed at the best-performing value (0.1), a final `GradientBoostingRegressor` was compared directly against a single `DecisionTreeRegressor` on the same train/test split. The single tree

was given a deeper maximum depth than the shallow trees used inside the boosting ensemble, since it's the only tree available to it:

```
gb_final = GradientBoostingRegressor(  
    n_estimators=100, learning_rate=0.1, max_depth=2, random_state=42  
)  
gb_final.fit(X_train, y_train)  
  
single_tree = tree.DecisionTreeRegressor(max_depth=5)  
single_tree.fit(X_train, y_train)  
  
for model in [single_tree, gb_final]:  
    preds = model.predict(X_test)  
    rmse = root_mean_squared_error(y_test, preds)  
    r2 = r2_score(y_test, preds)
```

Results on the test set:

Model	RMSE (\$ thousand)	R ²
Single decision tree	3.55	0.401
Gradient boosting (100 rounds, eta = 0.1)	2.04	0.802

Gradient boosting cuts RMSE roughly in half and roughly doubles the R² score compared to a single decision tree trained on the same data.

Visualizing the comparison. A scatter plot placed actual price on one axis and predicted price on the other, with a diagonal reference line representing the ideal case where predicted price exactly equals actual price. Predictions from the single decision tree were plotted in one color and predictions from gradient boosting in another. The gradient boosting points clustered much closer to the ideal diagonal line, while the single-tree points were noticeably more spread out and further from it. This visually confirms the same conclusion shown by the RMSE and R² numbers.

Common questions: XGBoost, bias-variance, and choosing settings

A few points clear up frequent points of confusion:

- Test data has no role during boosting. Every residual computation and every correction tree is fit purely on the training data. Test data is used only once, at the very end, when the fully-assembled final model (the sum of all rounds) generates predictions to measure performance.
- Only two ensemble techniques have a fixed name — random forest and gradient boosting. Any other combination is a valid ensemble a developer can design, but it has no established name of its own.
- Gradient boosting and XGBoost. XGBoost ("Extreme Gradient Boosting") is conceptually the same algorithm as gradient boosting, with only minor implementation and hyperparameter differences — an advanced, optimized version of the same underlying idea.
- A "prediction tree" is just a decision tree with a different target. The tree built at each boosting round is an ordinary decision tree algorithm. What changes is only which columns are used as the X and Y for training it: the original features plus the residual produced by the previous round's model, instead of the original target.
- No fixed formula exists for choosing the learning rate or the base algorithm. Both are experimental decisions — trying different values or algorithms and comparing performance, rather than computing them from a closed-form rule. A more complex algorithm, such as a neural network, is generally expected to produce better results than a simpler one, but this isn't guaranteed. Existing libraries already provide the algorithms, so the developer's job is to try several settings, compare results, and pick whichever performs best for the specific problem — not to write new algorithm implementations from scratch.

- Bias and variance. Unlike random forest, where each tree only sees a sample of the training data, every tree in a gradient boosting sequence trains on the complete training data. This creates a real possibility that an individual tree performs very well on training data but poorly on unseen test data — a variance problem. Each round is explicitly built to reduce the previous round's error on the training data itself, which can make that variance risk more pronounced. Using multiple models rather than one, and combining their outputs rather than relying on a single model's decision, can help address bias and variance. This is an inherent side effect of the ensemble structure rather than something the formulation explicitly targets or guarantees.

Watch out for: the round-by-round formula builds up exactly the same way each time —

`Prediction(round m) = Prediction(round m - 1) + eta × DT(X, residual of round m - 1)`. With 100 rounds of gradient boosting, this expands to residuals

computed all the way up through round 99, so it's easy to lose track of which residual belongs to which round if you're not careful with indexing.

3. Key Takeaways

- Gradient boosting is a sequential ensemble technique: each new model is trained to correct the residual (actual minus predicted) left behind by the combined model before it.
- The final prediction is a running sum: the naive starting average plus every correction tree's output, each scaled down by the learning rate (eta).
- The learning rate trades off speed and stability. Too small causes slow progress, too large overshoots the optimum, and a moderate value (0.1 by default) balances both.

- Random forest votes among independently trained trees; gradient boosting chains models sequentially and is algorithm-agnostic, unlike random forest's fixed reliance on trees.
- On a 150-row synthetic car dataset, `GradientBoostingRegressor` with a moderate learning rate roughly doubled the R^2 score and halved the RMSE of a single decision tree.

Mental model: Think of gradient boosting as a relay team correcting a rough first guess – each new runner (model) looks only at how far off the team is so far and nudges the total forward by a small, controlled step, rather than any single runner trying to sprint the whole distance alone.